

Firmware em C++ para o ESP32

1. Bibliotecas Usadas:

- **<Arduino.h>**: O framework base para controle dos pinos, portas seriais e interrupções do ESP32.
- **"arduinoFFT.h"**: Uma biblioteca matemática otimizada para o cálculo da Transformada Rápida de Fourier (FFT). É ela que permite ao microcontrolador converter os dados do "domínio do tempo" (a forma da onda) para o "domínio da frequência", revelando quais harmônicas e ruídos estão escondidos dentro do sinal elétrico.

2. Vetores de Onda (Matlab para Simulação)

Como o simulador Wokwi não possui suporte nativo para injetar uma rede AC de 220V com ruídos complexos, a solução de engenharia foi embarcar "gêmeos digitais" gerados no MATLAB diretamente na memória do ESP32.

- **senal_rede_Limpo, sinal_rede_THD_Alto, sinal_rede_Ruido_Alto**: São matrizes (arrays) com 128 posições. Cada número representa o valor exato que o conversor analógico-digital (ADC) do ESP32 leria a cada passo do tempo, já contemplando a conversão para 12-bits (valores de 0 a 4095) e o "offset" de proteção (nível DC de 1.65V). O ESP32 "lê" esses arrays como se estivesse lendo um sensor físico.

3. Parâmetros do Sistema

O código define as constantes físicas e os limites normativos de qualidade de energia:

- **V_NOMINAL = 220.0**: A tensão de referência do projeto.
- **Limites (LIMITE_SWELL, LIMITE_SAG, LIMITE_SPIKE)**: Baseados em normas técnicas, estabelecem que qualquer tensão acima de 242 V (+ 10%) é um *Swell* (sobretensão) e abaixo de 198 V (- 10%) é um *Sag* (afundamento). O *Spike* é parametrizado como um transiente perigoso de curtíssima duração (ex: 450 V).

4. Configuração do Pino Receptor da Interrupção

- **PINO_ZCD = 34**: Este pino recebe o sinal do Detector de Passagem por Zero.
- **detectaZeroCrossing()**: É uma função especial (Interrupção de Hardware - ISR) marcada com **IRAM_ATTR** para rodar na memória ultrarrápida do ESP32. Quando o pino 34 recebe um pulso (a onda da rede cruzou o zero), essa função congela o processador por nanossegundos, anota o tempo exato (**micros()**) e calcula o **periodoAtual** da onda elétrica.

5. Inicialização do Algoritmo de Fourier (FFT)

O sistema configura a FFT para analisar "janelas" de tempo:

- **amostrasFFT = 128**: Define o tamanho da janela (obrigatoriamente uma potência de 2).
- **vReal e vImag**: Vetores que armazenarão as amostras colhidas para o cálculo. A frequência de amostragem configurada no objeto FFT é de **7680 Hz** (128 amostras x 60 Hz), essencial para que as "caixas" de frequência (os *Bins*) representem os valores corretos matematicamente.

6. void setup()

A rotina de inicialização rodada uma única vez quando o ESP32 liga:

- Inicia a **Comunicação Serial** a 115200 bps para falar com o Python.
- Configura os **Pinos de Saída** (LEDs de alerta) e garante o funcionamento seguro de pinos conflitantes com a memória flash.
- Ativa a **Interrupção (attachInterrupt)**: Configura o pino do ZCD para disparar a função na borda de subida (RISING), ancorando o sincronismo do sistema.

7. void loop()

O loop principal foi desenhado para rodar sem atrasos fatais (delay bloqueantes), garantindo que a amostragem do sinal e a comunicação com a interface serial ocorram em tempo real.

A. Detecção de Sincronismo e Frequência

Quando a interrupção avisa que um ciclo novo começou (**novaLeituraZCD == true**), o loop pausa rapidamente outras interrupções (**noInterrupts()**) para copiar o tempo sem risco de

corrupção de memória.

- Calcula a **Frequência Real** (inverso do período em microssegundos).
Calcula o **Salto de Fase**: Compara o período lido com o período teórico ideal (16.66 ms para 60 Hz). Se houver diferença, converte esse atraso/adiantamento para graus. Se o desvio for muito grande, envia um ALERTA de sincronismo para a interface.

B. Máquina de Estados do Simulador (Injeção de Anomalias)

Lê qual onda deve ser usada no momento (Limpo, THD, ou Ruído) baseando-se nos comandos do painel.

- **Injeção Dinâmica**: Se clicar no botão "Gerar Swell", o código multiplica instantaneamente toda a amplitude da onda por 1,2. Se pedir "Sag", multiplica por 0,8. Se pedir "Spike", cria um pulso agressivo (1.5 x) apenas no semiciclo positivo.

C. Aquisição Analógica e RMS (Amostra por Amostra)

Este Bloco roda a cada iteração do loop, ponto a ponto.

- Guarda a amostra atual no vetor `vReal` para uso futuro pela FFT.
- Verifica imediatamente se a tensão daquele exato milissegundo rompeu o limite de *Spike* (o único distúrbio que não pode esperar o ciclo acabar para ser avisado).
Acumulador RMS: Eleva a amostra ao quadrado e soma na variável `somaQuadrados` (V^2), preparando o cálculo da tensão RMS.

D. Fechamento de Ciclo (Análise da Janela Completa)

Quando o sistema atinge as 128 amostras, o ciclo elétrico de 60 Hz se fecha e os cálculos mais pesados começam:

- **Finalização do RMS**: Tira a raiz quadrada da média da `somaQuadrados` e acende os LEDs de Swell ou Sag físicos se necessário.
- **Execução da FFT**: Roda as funções *windowing* (suavização da onda), *compute* (transformada) e *complexToMagnitude* (conversão polar).
- **Análise THD**: Lê a energia exatamente nas gavetas (bins) das frequências 180 Hz, 300 Hz e 420 Hz (3ª, 5ª e 7ª harmônicas), aplica a fórmula e calcula a Distorção Harmônica Total.
- **Filtro EMI**: Varre as altas frequências (Bins 15 a 63) somando a energia residual para calcular se há ruído de alta frequência.
- Zera os acumuladores para o próximo ciclo elétrico.

E. Comunicação com Interface

Gerencia o tráfego de dados com o cabo USB:

- **Transmissão**: Envia o ponto atual da onda (`A:0:valor`) para animar o osciloscópio do Python.
- **Recepção**: Verifica se a porta `Serial.available()` tem dados recebidos do dashboard. Se o Python enviou, por exemplo, o texto `"D:13:1"`, o ESP32 divide a string, identifica que é um Comando Digital para ativar o Botão de "Swell" (Pino 13 virtual) e altera a *flag* booleana correspondente, modificando o comportamento da simulação instantaneamente.